Universidade Estadual de Maringá – Departamento de Informática

Engenharia de Software · Introdução à Inteligência Artificial

Análise de artigos científicos com Processamento de Linguagem Natural

Trabalho 2 · Tema: Processamento de Linguagem Natural (IEEE Xplore)

Equipe

Felipe Matheus – RA 107496

Gabriel Balancieri Perassoli – RA 135854

Guilherme da Silva Nascimento – RA 130162

Professor

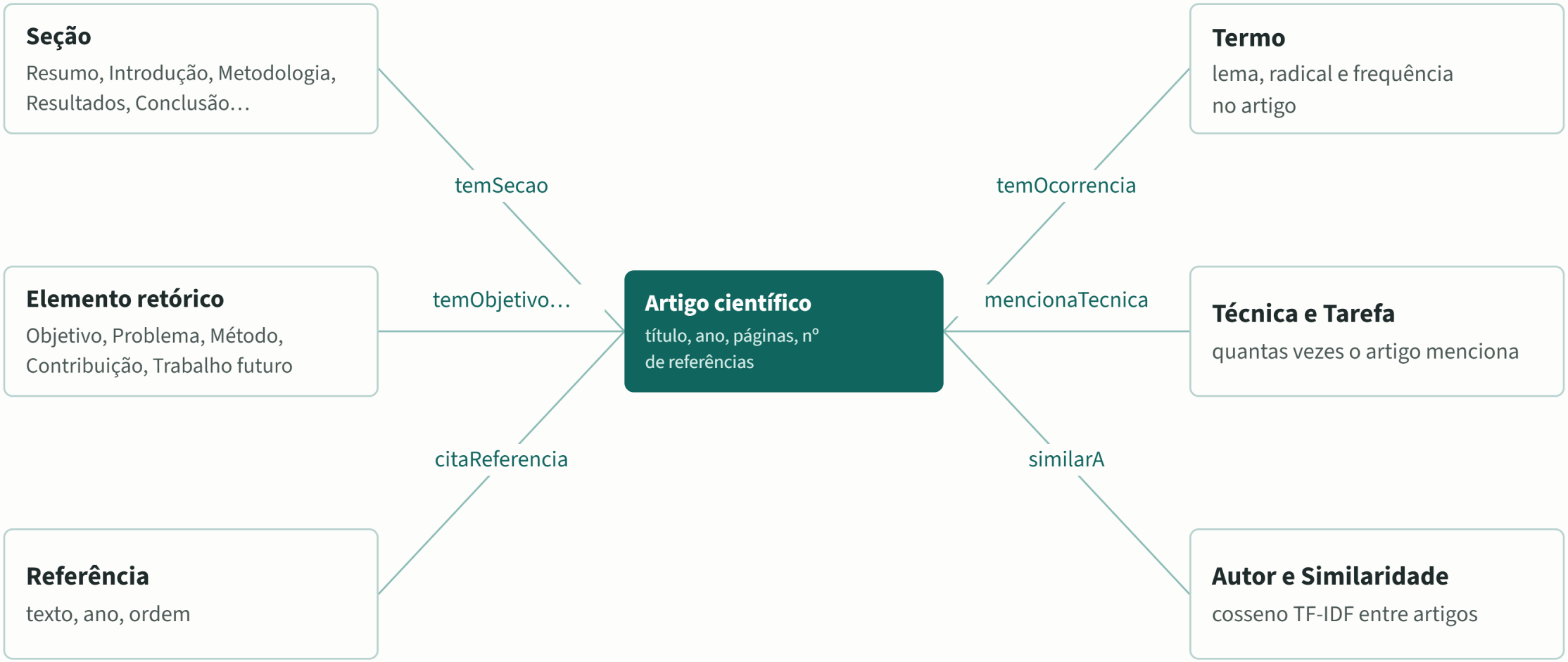
Wagner Igarashi

Introdução

- Para revisar a literatura de um tema é preciso ler muitos artigos e anotar, em cada um, o objetivo, o problema, o método e as contribuições.
- Proposta: um programa que lê os PDFs dos artigos e extrai essas informações automaticamente.
- O programa também encontra os termos mais citados e as referências, e salva tudo numa ontologia.
- Restrição da especificação: nenhuma biblioteca de aprendizado de máquina, só regras e estatística.

Tema: Processamento de Linguagem Natural · **Base:** IEEE Xplore

Modelagem: ontologia do artigo científico



Fundamentação

Técnica	Para que foi usada
Tokenização e stopwords	separar as palavras e retirar as que não têm significado (the, of, and)
Lematização e stemming	reduzir à forma do dicionário (studies → study) e ao radical (extraction → extract)
Bag-of-words e n-gramas	contar palavras e sequências de duas e três palavras
TF-IDF e cosseno	dar peso aos termos e medir a semelhança entre artigos
Modelo de bigramas	probabilidade da próxima palavra, com suavização de Laplace
Regras com frases-gatilho	achar objetivo, problema, método e contribuição (Teufel e Moens, 2002)
Ontologia (RDF/OWL)	representar o artigo com classes e propriedades; Turtle, JSON-LD e SPARQL

Materiais

Item	Utilizado
Linguagem	Python 3
Ambiente	linha de comando (python main.py), em qualquer sistema operacional
Leitura de PDF	pypdf (sucessor do PyPDF2) e pdfplumber
PLN	NLTK: apenas stopwords, WordNetLemmatizer e PorterStemmer
Ontologia	rdflib (Turtle, JSON-LD e SPARQL)
Gráficos	matplotlib, wordcloud, networkx e numpy
Biblioteca padrão	re, collections, math, json, argparse, unicodedata
Não usadas	spaCy, scikit-learn, Transformers, TensorFlow, PyTorch, Word2Vec (proibidas)

Método: etapas do programa

1. Leitura

texto com pypdf; remove cabeçalho, rodapé e hifenização; divide em seções

2. Pré-processamento

tokens, stopwords, lemas e radicais

3. Modelos

bag-of-words, n-gramas, TF-IDF e bigramas

4. Extração

termos, referências, objetivo, problema, método, contribuição e trabalho futuro

5. Saídas

ontologia, relatório, gráficos e avaliação

- Etapa 1 da especificação: passos 1 a 3 e a parte de termos e referências do passo 4.
- Etapa 2: objetivo, problema, método, contribuições e trabalhos futuros (passo 4).
- Etapas 3 e 4: ontologia em Turtle e JSON-LD e avaliação com gabarito (passo 5).

Etapa 2: extração por regras

Cada frase recebe pontos para cada elemento: **+3** por gatilho forte, **+1** por gatilho fraco e um bônus ou desconto pela seção. A frase fica no elemento em que passou do limite mínimo com mais folga.

Elemento	Exemplos de gatilhos fortes	Seções com bônus
Objetivo	“the goal of this paper”, “in this paper”, “this work aims to”	Resumo, Introdução
Problema	“however”, “challenging”, “expensive”, “lack”, “fail”	Resumo, Introdução
Método	“we use / train / conduct”, “our approach uses”	Resumo, Introdução
Contribuição	“contributes”, “to our knowledge… the first”, itens da lista de contribuições	Introdução, Conclusão
Trabalho futuro	“future work”, “we plan to”, “would be interesting”	Conclusão

Objetivo × contribuição: se a frase passa no limite de contribuição, ela é contribuição e não pode ser objetivo. Frases como “equal contribution” perdem pontos.

Fonte dos dados

12 artigos de PLN publicados entre 2005 e 2023 (129 páginas e 408 referências).

Ano	Artigo	Evento
2005	A Statistical Semantic Parser that Integrates Syntax and Semantics	CoNLL
2007	Sparse Information Extraction: Unsupervised Language Models to the Rescue	ACL
2007	Learning to Extract Relations from the Web using Minimal Supervision	ACL
2009	Efficient Parsing of Syntactic and Semantic Dependency Structures	CoNLL
2011	Identifying Relations for Open Information Extraction	EMNLP
2012	Joint Feature Selection in Distributed Stochastic Learning… in SMT	ACL
2013	Discriminative Joint Modeling of Lexical Variation and Acoustic Confusion…	NAACL
2014	How to make words with vectors: Phrase generation in distributional semantics	ACL
2018	Construction of the Literature Graph in Semantic Scholar	NAACL
2020	SPECTER: Document-level Representation Learning using Citation-informed Transformers	ACL
2020	S2ORC: The Semantic Scholar Open Research Corpus	ACL
2023	LongEval: Guidelines for Human Evaluation of Faithfulness in Long-form Summarization	EACL

Etapa 1: termos mais citados

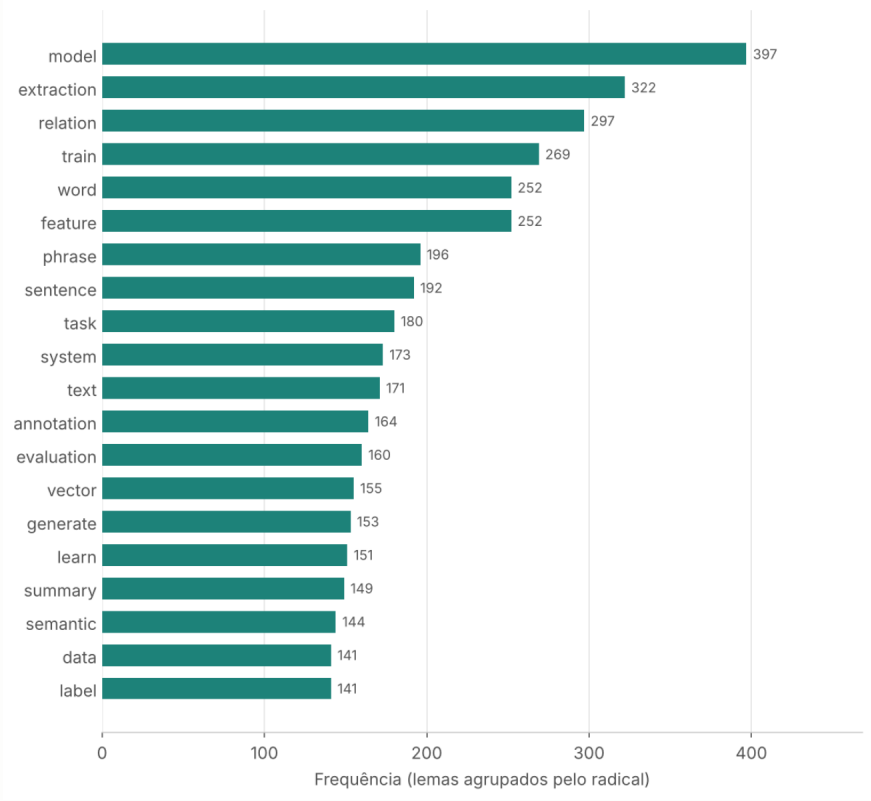


Figura 1 – Os 20 termos mais citados nos 12 artigos

- 10 mais citados: model, extraction, relation, train, word, feature, phrase, sentence, task e system.
- Contagem feita sem as referências, sem stopwords e com os lemas agrupados pelo radical: extraction + extract + extractive = 322.
- Bigramas mais frequentes: relation phrase (86), language model (65) e human evaluation (48).

Etapa 1: referências bibliográficas

- Três formatos reconhecidos: IEEE ([1], [2]…), lista numerada e autor-ano (ACL, APA).
- 407 de 408 referências separadas corretamente nos 12 artigos (99,8%) e 210 de 210 nos 3 adicionais.
- As referências de cada artigo ficam em saida/referencias, uma por linha.

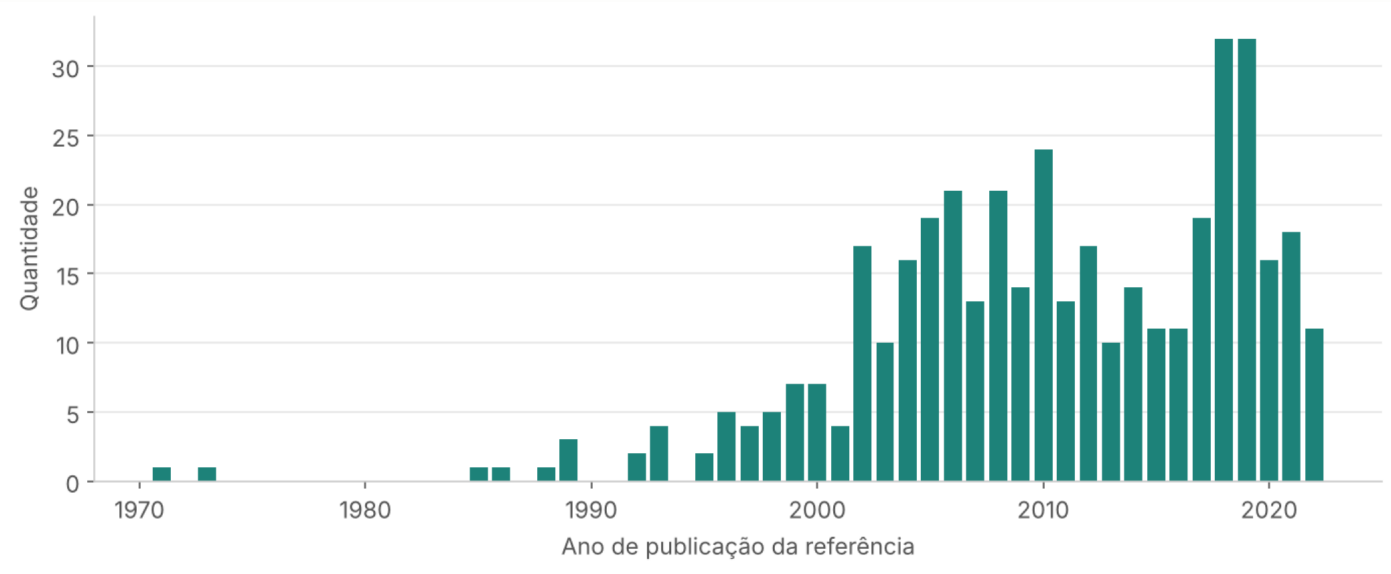


Figura 2 – Ano de publicação das referências citadas pelos 12 artigos

Nuvem de palavras geral



Figura 3 – Tamanho da palavra proporcional à frequência

- As palavras maiores repetem o ranking da Figura 1.
- Termos como summary, annotation e evaluation vêm dos artigos mais recentes.

Técnicas e tarefas mais mencionadas

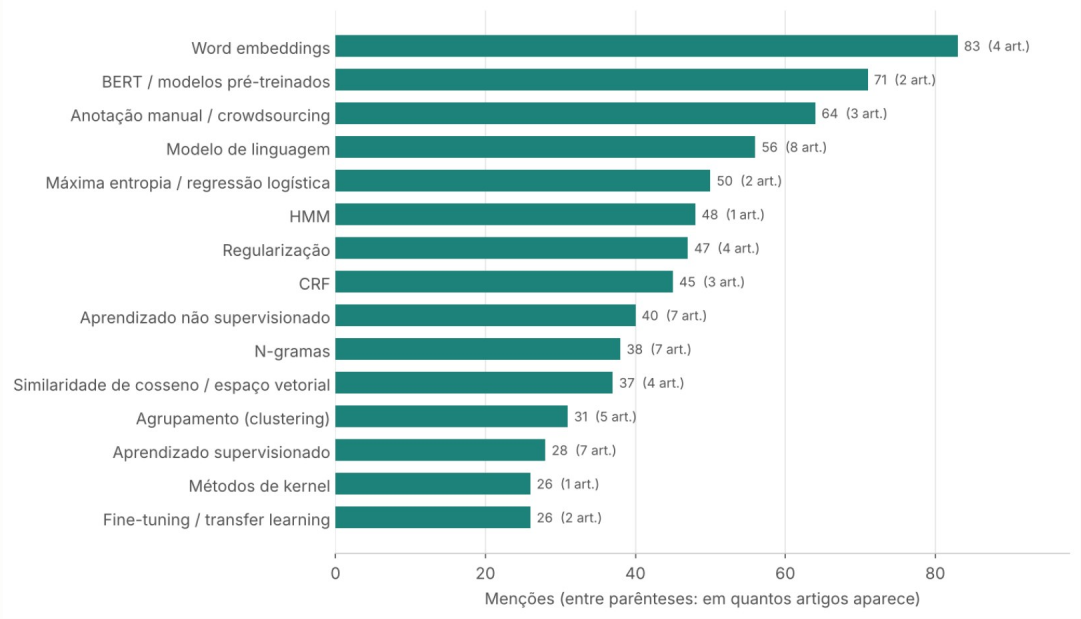


Figura 4 – Técnicas (entre parênteses, em quantos artigos aparecem)

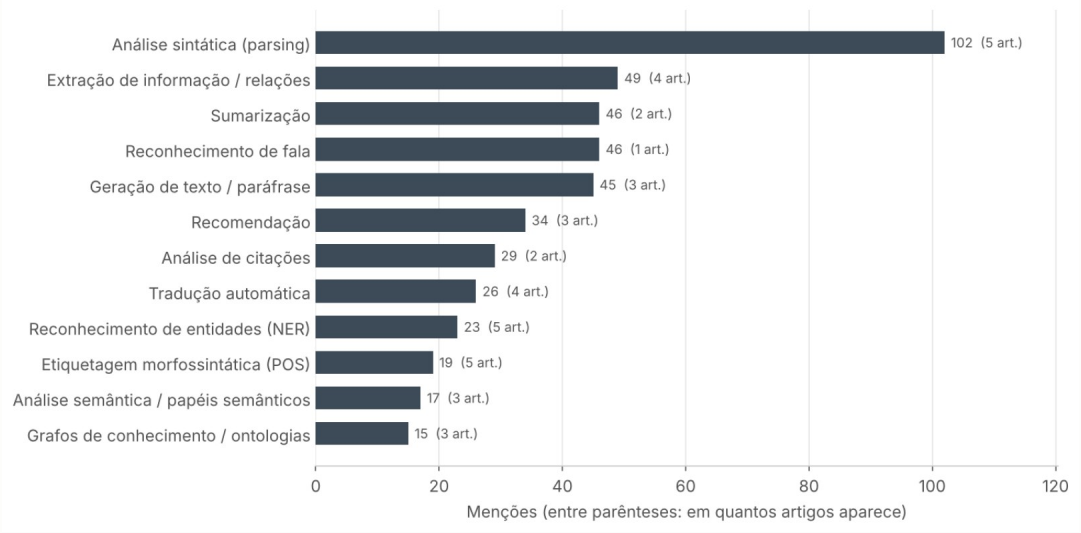


Figura 5 – Tarefas de PLN

- Modelo de linguagem é a técnica presente em mais artigos (8 de 12); análise sintática é a tarefa mais citada.

Evolução temporal dos termos

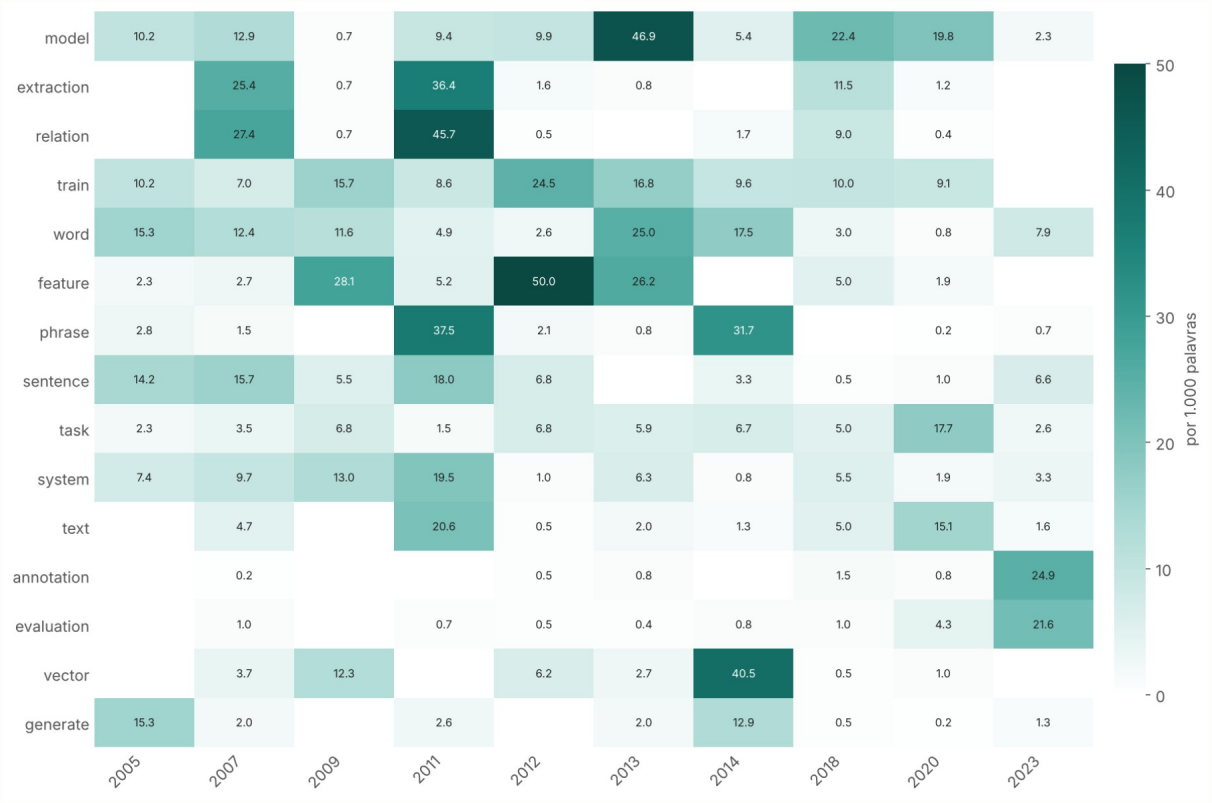


Figura 6 – Ocorrências por 1.000 palavras em cada ano

- relation e extraction aparecem mais entre 2007 e 2011.
- vector se destaca em 2014; annotation e evaluation, em 2023.
- Nas técnicas: HMM só aparece em 2007, e BERT e Transformer só em 2020.
- Como há quase um artigo por ano, o assunto de cada artigo pesa bastante.

Termos que aparecem como trabalhos futuros

20 frases de trabalho futuro encontradas nos 12 artigos, quase todas na conclusão. Os assuntos mais comuns: melhorar os modelos, usar mais informação e literatura e tratar viés.

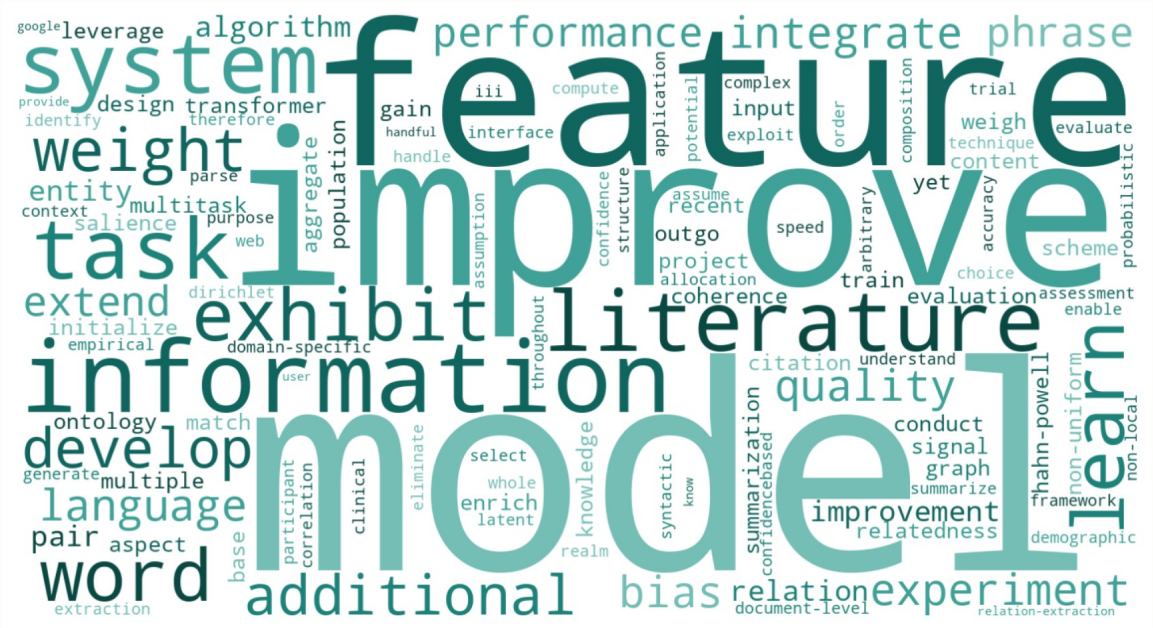


Figura 7 – Nuvem dos trabalhos futuros

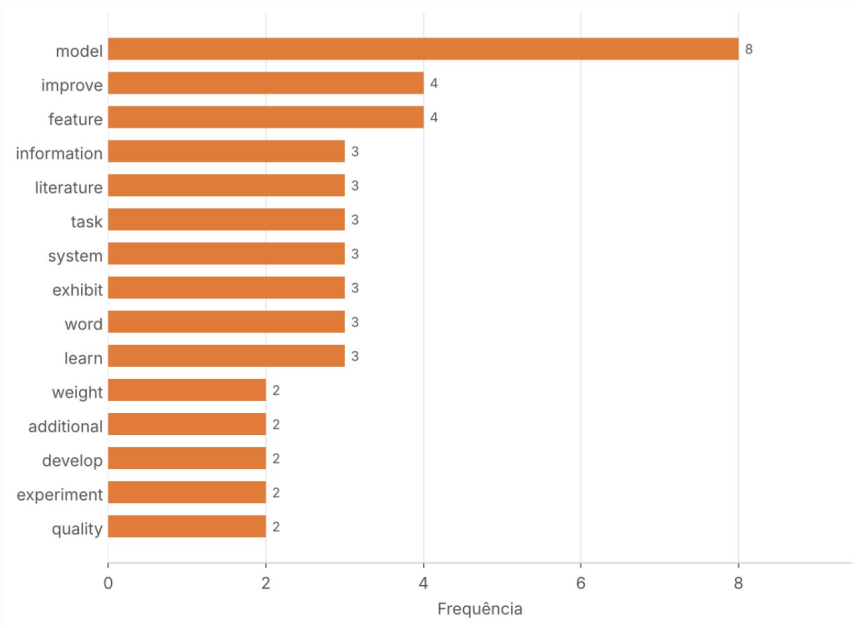


Figura 8 – Termos mais frequentes

Coocorrência de termos e similaridade entre artigos

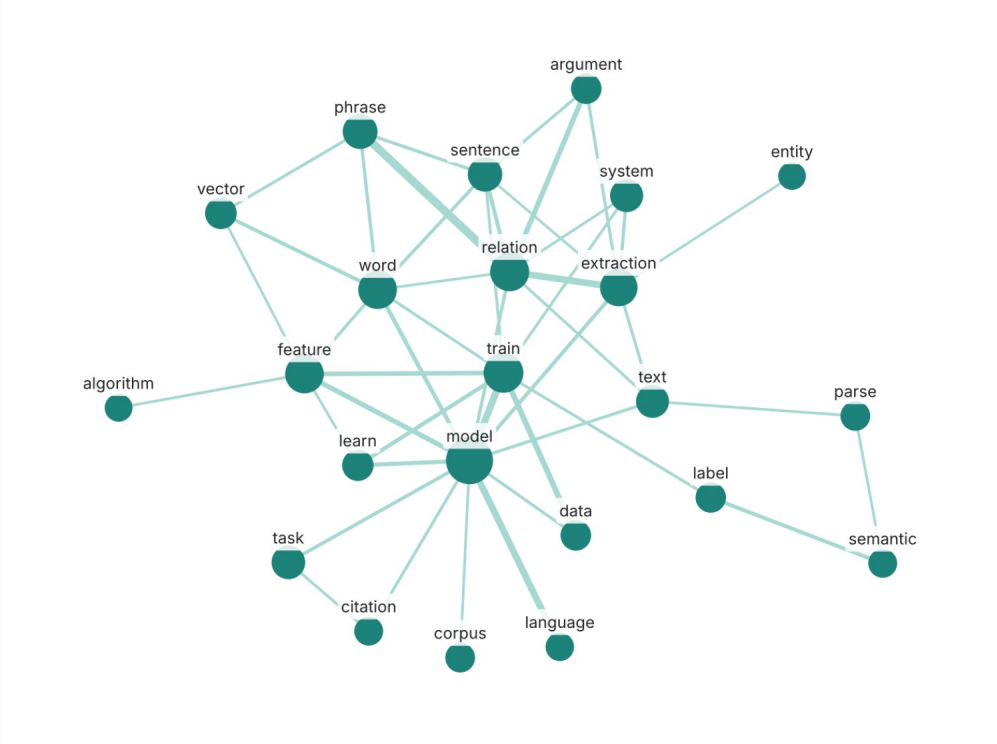


Figura 9 – Termos na mesma sentença. Par mais forte: relation e phrase (82 sentenças)



Figura 10 – Cosseno TF-IDF. Mais parecidos: Sparse IE e ReVerb, SPECTER e S2ORC (0,31)

Árvore de palavras e modelo de bigramas

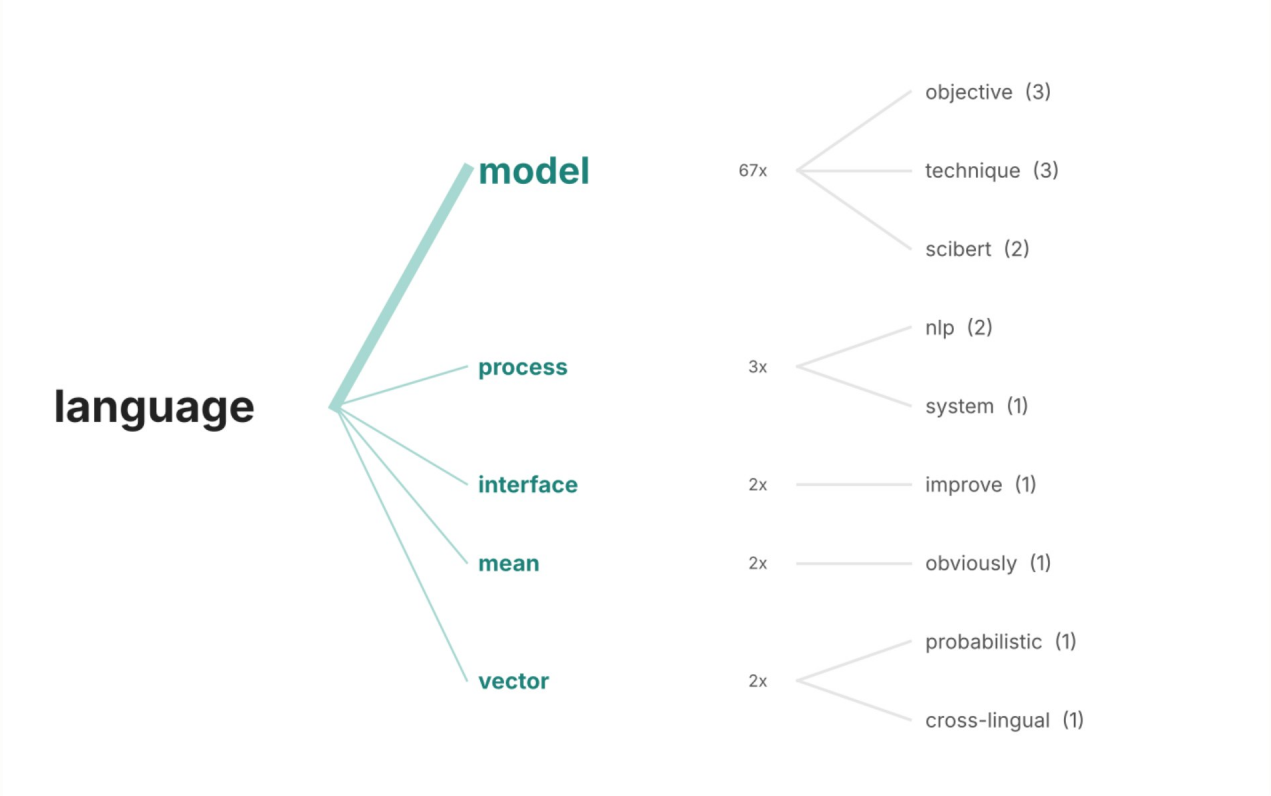


Figura 11 – Palavras que mais aparecem depois de “language”

- “language model” aparece 67 vezes.
- A árvore usa as contagens de bigramas e trigramas.
- O modelo de bigramas, treinado com 11 artigos e testado no restante, deu perplexidade entre 2.475 e 3.042.

Etapa 3: dados salvos como ontologia

saida/ontologia_artigos.ttl (trecho)

```
art:artigo_fader11 a art:ArtigoCientifico ;
    art:ano "2011"^^xsd:gYear ;
    art:numeroReferencias 30 ;
    art:temObjetivo art:artigo_fader11_objetivo_1 ;
    art:temContribuicao art:artigo_fader11_contribuicao_4 .

art:artigo_fader11_contribuicao_4 a art:Contribuicao ;
    art:extraidoDe art:artigo_fader11_secao_7 ;
    art:pontuacao 6 ;
    art:texto "We make REVERB and the data used in
        our experiments available…"@en .
```

- Formatos: Turtle (.ttl) e JSON-LD (.jsonld), com o mesmo conteúdo.
- 5.459 triplas para os 12 artigos.
- Ontologia base com 27 classes e 37 propriedades.
- O script consultar_ontologia.py faz consultas SPARQL, como “técnicas citadas em mais artigos”.

Demonstração com 3 artigos novos

```
python main.py --pasta artigos_adicionais --saida saida_adicionais --avaliar
```

Artigo	Ano	Referências	Termos principais
Paper Plain	2022	111 de 111	read, plain, participant
Multi-LexSum	2022	64 de 64	summary, summarization
Beyond Summarization	2023	35 de 35	write, author, support

- Nenhum dos 3 artigos foi usado para ajustar as regras.
- Título e ano corretos nos 3; tempo total de cerca de 8 segundos.

Exemplo de extração: Multi-LexSum (2022)

Elemento	Trecho extraído	No gabarito?
Objetivo	“In this paper, we present a new dataset, Multi-LexSum, for studying automatic summarization…”	Sim
Objetivo	“We present extensive analysis demonstrating that despite the high-quality summaries…”	Não
Problema	“Multi-LexSum presents a challenging multi-document summarization task…”	Sim
Problema	“With the advent of large language models, methods for abstractive summarization…”	Não
Método	“We conduct a series of experiments on Multi-LexSum, and find that existing summarization models perform poorly.”	Sim
Contribuição	“But to our knowledge, Multi-LexSum is the first dataset to provide summaries at multiple levels of granularity.”	Sim

Objetivo e contribuição ficaram separados: a frase “to our knowledge… the first” foi para contribuição.

Etapa 4: como avaliamos

- Gabarito feito à mão para 15 artigos: objetivo, problema, método, contribuição, trabalho futuro, título, ano e número de referências.
- 12 artigos foram usados para ajustar as regras; os 3 adicionais só no teste.
- Uma frase extraída é acerto quando o F1 entre as palavras dela e as de uma frase do gabarito é pelo menos 0,5 (como no ROUGE-1).
- Precisão = acertos ÷ frases extraídas; revocação = acertos ÷ frases do gabarito; F1 = média harmônica das duas.
- Também medimos as referências, o título, o ano e o tempo por artigo.

Resultados da avaliação

Medida	12 artigos	3 novos
Objetivo (F1)	0,70	0,67
Problema (F1)	0,38	0,67
Método (F1)	0,55	0,57
Contribuição (F1)	1,00	1,00
Trabalho futuro (F1)	0,93	0,57
Média (F1)	0,71	0,70
Referências	99,8%	100%
Título e ano	100%	100%
Tempo por artigo	0,6 s	1,5 s

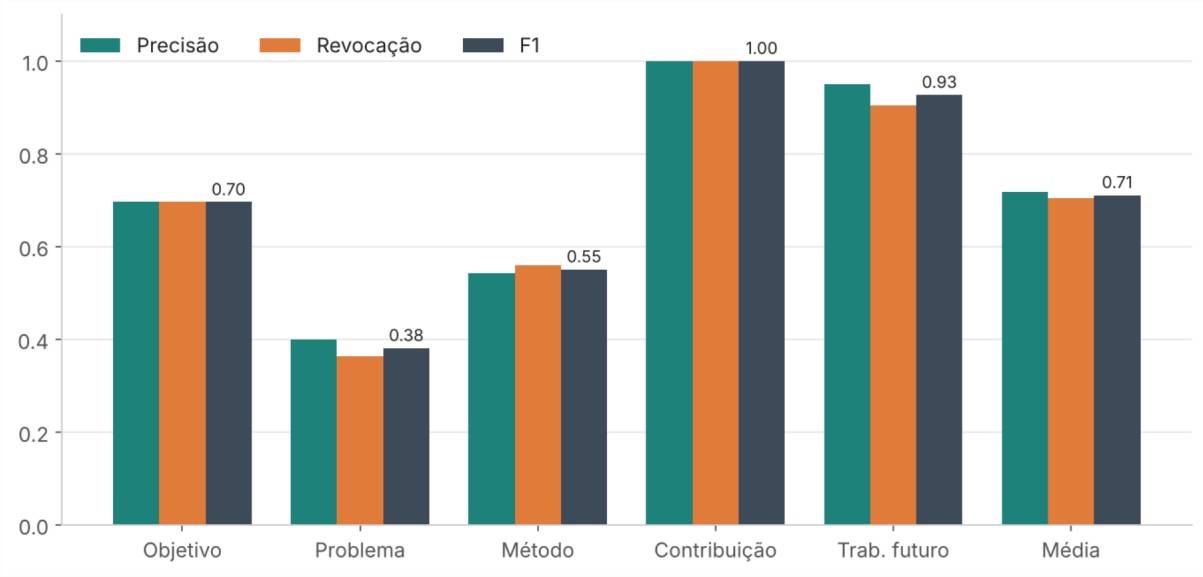


Figura 12 – Precisão, revocação e F1 nos 12 artigos

Análise da avaliação

- Contribuição (F1 1,00) e trabalho futuro (0,93) são os mais fáceis: os autores usam expressões fixas, como “our contributions” e “future work”.
- Problema é o mais difícil (0,38): aparece espalhado em várias frases e nem sempre com palavras negativas. O sistema encontra a ideia, mas escolhe outra frase.
- Método (0,55): o gabarito tem até 3 frases por artigo e parte delas não usa os gatilhos esperados.
- Objetivo e contribuição ficaram separados: das 34 frases dessas duas categorias no gabarito, só 2 foram trocadas.
- Nos artigos novos o F1 médio quase não mudou (0,71 para 0,70). Trabalho futuro caiu porque o sistema pegou recomendações de design como se fossem trabalho futuro.

Conclusões

- Com regras e estatística, sem aprendizado de máquina, a extração chegou a F1 médio de 0,71.
- Referências, título e ano foram extraídos praticamente sem erros.
- A ontologia permite consultar os resultados com SPARQL.
- Limitações: as regras foram feitas para artigos em inglês, tabelas e figuras no meio do texto geram ruído e o gabarito foi anotado por uma só pessoa.
- Melhorias possíveis: mais regras para problema e método, mais anotadores e mais artigos por ano na análise temporal.

Bibliografia

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing*. 3. ed. (rascunho on-line), 2025.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

BIRD, S.; KLEIN, E.; LOPER, E. *Natural Language Processing with Python*. O'Reilly, 2009.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 4. ed. Pearson, 2020.

PORTER, M. F. An algorithm for suffix stripping. *Program*, v. 14, n. 3, p. 130–137, 1980.

MILLER, G. A. WordNet: a lexical database for English. *Communications of the ACM*, v. 38, n. 11, p. 39–41, 1995.

TEUFEL, S.; MOENS, M. Summarizing scientific articles: experiments with relevance and rhetorical status. *Computational Linguistics*, v. 28, n. 4, p. 409–445, 2002.

PERONI, S.; SHOTTON, D. The SPAR Ontologies. In: *ISWC 2018*. LNCS 11137. Springer, 2018.

W3C. *RDF 1.1 Turtle* (2014); *OWL 2 Web Ontology Language* (2012); *JSON-LD 1.1* (2020).

Documentação oficial: pypdf, pdfplumber, NLTK, rdflib, matplotlib, wordcloud e networkx.

Os 12 artigos analisados e os 3 artigos da demonstração.